Esercitazione di Laboratorio 04S

Temi trattati

1. Liste di elementi
2. Tabelle
3. Funzioni e metodi
4. Ripasso

Discussione

A. Discutere somiglianze e differenze tra liste e tabelle.
B. Descrivere l'uso degli indici in:
    a. stringhe;
    b. liste;
    c. tabelle
C. Come si previene un `out-of-range` error?
D. Quale è l'algoritmo di ricerca che serve per trovare un elemento in un insieme di elementi non ordinato?
E. Quale costrutto è il più adatto per scorrere i valori di una tabella con righe e colonne?

Esercizi

***Consegna**: per ciascuno degli esercizi seguenti, scrivere un programma in Python che risponda alle richieste indicate. Completare gli esercizi seguenti in aula.*

04S.1 Tabella. Scrivere le istruzioni Python per eseguire le seguenti operazioni con una tabella di `m` righe e `n` colonne (dimensioni inserite da tastiera):

I. inizializzare la tabella con valori pari a zero (`0`);
II. riempire tutte le caselle con valori pari a uno (`1`);
III. riempire le caselle alternando `0` e `1` in uno schema a scacchiera;
IV. riempire di `0` solo le caselle della riga superiore e di quella inferiore, lasciando invariato il resto della tabella;
V. riempire con `1` solo le caselle della colonna di destra e di sinistra, lasciando invariato il resto della tabella;
VI. calcolare e stampare la somma di tutti gli elementi.

Dopo ogni operazione, stampare la tabella. [Ex 08.1.4]

04S.2* Distanziamenti. Le persone che parcheggiano la propria automobile in una fila di parcheggi di solito preferiscono massimizzare la distanza tra il posto che occupano e i posti che sono già occupati da altri veicoli. Tendono quindi ad occupare il posto centrale della fila più lunga di posti liberi a disposizione. Ad esempio, si consideri la situazione in cui dieci posti sono liberi:

`_ _ _ _ _ _ _ _ _ _`

La prima persona che arriva occuperà, col proprio veicolo, un posto nella parte centrale della fila:

`_ _ _ _ _ X _ _ _ _`

La persona successiva lo posizionerà a metà della fila lasciata libera più lunga (cioè quella di sinistra):

`_ _ X _ _ X _ _ _ _`

Scrivere un programma che riceva in input il numero di posti auto di cui si compone la fila di

parcheggi e che, ogni volta che un nuovo posto viene occupato secondo la regola indicata, visualizzi la fila nel formato indicato sopra. Suggerimento: utilizzare un elenco di valori booleani per indicare se un posto auto è occupato o meno. [Ex 07.2.2]

04S.3 Giocare a dadi. Scrivere un programma che generi una sequenza di 20 lanci casuali di dadi, li memorizzi in una lista e visualizzi i valori generati, contrassegnando la serie di valori identici più lunga con questa formattazione:
`1 2 5 5 3 1 2 4 3 (2 2 2 2) 3 6 5 5 6 3 1`
Se sono presenti più sequenze di valori identici di lunghezza massima, si usi la formattazione indicata per evidenziare la prima in ordine di occorrenza. [Ex 08.1.3]

04S.4 Unione di liste ordinate. Scrivere la funzione `merge_sorted(a, b)` che unisca due liste (che si assumono già ordinate in modo crescente), restituendo una nuova lista ordinata in modo crescente. Gestire un indice corrente per ciascuna lista, in modo da tenere traccia delle porzioni già elaborate. Le liste di partenza non devono essere modificate. Se, ad esempio, il contenuto di `a` è `1 4 9 16` e il contenuto di `b` è `4 7 9 9 11`, l'invocazione `merge_sorted(a, b)` restituisce una nuova lista contenente i valori `1 4 4 7 9 9 9 11 16`. Non utilizzare il metodo `sort()` né la funzione `sorted()`. [Ex 08.1.6]

*Per casa: rifare l'esercizio **03S.1 Lista di numeri casuali** definendo una funzione per ciascun compito. Tutte le 5 funzioni dovranno ricevere come argomento la lista creata nel main con numeri casuali. Nel caso (V) la funzione deve restituire la lista ordinata, pronta per ulteriori manipolazioni nel main, non solo la visualizzazione a terminale. Inoltre, svolgere l'esercizio del Lab9 numero **09.2.1 A teatro**, si tratta di un vero e proprio esercizio di problem solving oltre che di programmazione. Da ora in poi nel rivedere gli esercizi svolti e nei nuovi esercizi proposti valutare sempre se e come le funzioni possano essere utilizzate per aumentare la leggibilità e praticità del codice.*